Lesson 5 Worksheet: Implementing the Manhattan Class — ANSWER KEY

Part 1: Code Tracing — compute_path() Step by Step

Trace the call: `Manhattan((0, 0)).compute_path((2, 3))`

Setup variables:

Variable	Initial Value
`self.position`	(0, 0)
`current_row`	0
`current_col`	0
`dest_row`	2
`dest_col`	3
`path` (initial)	[]

South while loop — trace each iteration:

Iteration	current_row (before)	< dest_row?	current_row (after)	Appended to path
1	0	YES	1	(1, 0)
2	1	YES	2	(2, 0)
Check	2	NO	— (loop ends)	—

North while loop: Does it execute? NO Why? current_row (2) is not greater than dest_row (2), so the condition is false.

East while loop — trace each iteration:

Iteration	current_col (before)	< dest_col?	current_col (after)	Appended to path
1	0	YES	1	(2, 1)

Iteration	current_col (before)	< dest_col?	current_col (after)	Appended to path
2	1	YES	2	(2, 2)
3	2	YES	3	(2, 3)
Check	3	NO	— (loop ends)	—

West while loop: Does it execute? NO Why? current_col (3) is not greater than dest_col (3), so the condition is false.

Final returned path: [(1, 0), (2, 0), (2, 1), (2, 2), (2, 3)]

len(path): 5

Part 2: Trace a Reverse Path

Trace the call: `Manhattan((3, 3)).compute_path((1, 0))`

Setup variables:

Variable	Initial Value
`current_row`	3
`current_col`	3
`dest_row`	1
`dest_col`	0

Which two while loops execute? North and West

North while loop:

Iteration	current_row (before)	current_row (after)	Appended
1	3	2	(2, 3)
2	2	1	(1, 3)

West while loop:

Iteration	current_col (before)	current_col (after)	Appended
1	3	2	(1, 2)
2	2	1	(1, 1)

Iteration	current_col (before)	current_col (after)	Appended
3	1	0	(1, 0)

Final returned path: [(2, 3), (1, 3), (1, 2), (1, 1), (1, 0)]

len(path): 5

Part 3: Fill in the Blanks

```
class Manhattan:
    def __init__(self, start):
        self.position = start

    def compute_path(self, destination):
        path = []

        current_row, current_col = self.position
        dest_row, dest_col = destination

        # Move south
        while current_row < dest_row:
            current_row = current_row + 1
            path.append((current_row,  current_col))

        # Move north
        while current_row > dest_row:
            current_row = current_row - 1
            path.append((current_row,  current_col))

        # Move east
        while current_col < dest_col:
            current_col = current_col + 1
            path.append((current_row,  current_col))

        # Move west
        while current_col > dest_col:
            current_col = current_col - 1
            path.append((current_row,  current_col))

        return path
```

Blanks in order:

1. `position`
2. `[]`
3. `position`
4. `destination`

5. `<`
6. `1`
7. `append`
8. `current_row`
9. `current_col`
10. `>`
11. `1`
12. `append`
13. `current_row`
14. `current_col`
15. `<`
16. `1`
17. `append`
18. `current_row`
19. `current_col`
20. `>`
21. `1`
22. `append`
23. `current_row`
24. `current_col`
25. `path`

Part 4: Predict the Output

Program A:

Line 1: [(0, 1), (0, 2), (0, 3)]

Line 2: Steps: 3

What kind of edge case is this? Same row – the south and north loops do not execute, so the path only has column movement (east).

Program B:

Line 1: []

Line 2: Steps: 0

Why is the path empty? The start equals the destination, so none of the four while loop conditions are true. No loops execute, and the path stays as the empty list [].

Program C:

Line 1: Path 1: [(1, 0), (2, 0), (2, 1), (2, 2), (2, 3)]

Line 2: Path 2: [(1, 0), (1, 1)]

Do both paths start from the same position? YES

Why? `compute_path` always uses `self.position` as the starting point, which is (0, 0) and never changes. The method does not update `self.position` – it only modifies local variables (`current_row` and `current_col`). So every call to `compute_path` starts from the original position (0, 0). The first step in each path is a move away from (0, 0).

Part 5: Find and Fix the Bug

Bug 1:

What is wrong? `path` starts as `[self.position]` instead of `[]` . The starting position should NOT be in the path (the robot is already there). This would add an extra element at the beginning, and `len(path)` would be one more than the actual number of steps.

Fix: Change `path = [self.position]` to `path = []`

Bug 2:

What is wrong? The west loop adds 1 instead of subtracting 1: `current_col = current_col + 1` . This makes the column go the wrong direction (east instead of west), and the loop condition `current_col > dest_col` stays true forever, creating an infinite loop.

What would happen if you called `compute_path((2, 3))` from start (3, 3)? The north loop runs correctly, moving row from 3 to 2. Then the west loop should move column from 3 to 2, but instead it adds 1 each time (3, 4, 5, 6, ...), creating an infinite loop that never terminates.

Fix: Change `current_col = current_col + 1` to `current_col = current_col - 1` in the west while loop.

Bug 3:

What is wrong? In the east while loop, the append call has row and column swapped: `path.append((current_col, current_row))` instead of `path.append((current_row, current_col))` . This puts the column value in the row position and vice versa.

If start is (0, 0) and destination is (2, 3), what would the INCORRECT last three elements of the path be?

(1, 2), (2, 2), (3, 2)

What should they be?

(2, 1), (2, 2), (2, 3)

Reflection

The compute_path() method uses 4 separate while loops instead of if/else statements with 2 while loops. Why does this simpler approach work? (Hint: think about what happens when a while loop's condition is already false.)

A while loop whose condition is already false simply does nothing – it is skipped entirely. So we can write all four directions (south, north, east, west) as separate loops without any if/else logic. For any given path, only the one or two loops whose conditions are true will actually execute. For example, going from (0,0) to (2,3), only the south loop (current_row < dest_row) and east loop (current_col < dest_col) run. The north and west loops are automatically skipped because their conditions are false. This is simpler than using if/else to choose between +1 and -1 step values.